OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment	:		Individual
Duration	:	1	Week
Total Marks	:		50
Submission Date	: 10/11/2026		

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192521288/SLOT-C-OOP.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Three parameter-passing methods

Method	Parameter represents	Effect on caller
Call by value	A local copy of the argument	Changes affect only the copy.
Call by reference	An alias for the caller variable	Changes affect the original variable.
Call by pointer	A copied address that points to caller variable	Dereferencing the pointer changes the original; pointer must be valid.

```
void concessionValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void concessionReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void concessionPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= *fee * percent / 100;
}
```

The value parameter is stored as a separate local object. A reference parameter is an alias to the caller object. A pointer parameter is a local pointer value containing the caller object address; dereferencing it accesses the original fee. Only reference and pointer versions modify the caller fee.

When to use const reference

Use const long long& or, more importantly, const Student& when a function needs to inspect an object without copying it and must not modify it. For a small built-in type such as int or long long, pass-by-value is usually simpler and inexpensive.

Design decision for multiple concessions

For a single fee update, returning the new fee is easy to reason about and supports chaining: fee = applyConcession(fee, percent). For a workflow applying several sequential discounts, an in-place reference can be convenient. The recommended interface here is a pure function returning the new fee because it makes the change explicit and avoids hidden mutation. Define whether discounts are sequential or mutually exclusive in the fee policy.

Prediction and exact output of supplied program

```
1 12 13\n1 100\n16 13 6
```

Explanation: change(a,b,&c) changes only local copy a, so caller a remains 1. b is a reference and increases from 2 to 12. c is reached through its pointer and increases from 3 to 13. bigger(a,c) returns a reference to c because c is greater; assignment sets c to 100. show(1) uses defaults y=5,z=10 and prints 16; show(1,2) uses z=10 and prints 13; show(1,2,3) uses no defaults and prints 6.

Completed TODO 1

```
void applyConcession(long long& fee, int percent) {
    if (percent < 0 || percent > 100) return;
```

```
    fee -= fee * percent / 100;
}
```

Expected output:

```
after call by value   : 45000\nafter call by reference: 40500
```

applyByValue receives a local copy of 45000, so its subtraction does not alter hostelFee. applyConcession receives a reference to hostelFee and subtracts 10%, leaving 40500.

Alternative return-value design and complexity

```
long long applyConcession(long long fee, int percent) {
    if (percent < 0 || percent > 100) return fee;
    return fee - fee * percent / 100;
}
// usage:
hostelFee = applyConcession(hostelFee, 10);
```

All three approaches take O(1) time for a scalar fee. Passing a long long by value copies one scalar (typically 8 bytes); passing by reference or const reference passes an address-like reference and avoids copying the referred object. Passing a large Student object by value copies all its members and may be expensive; reference/const reference avoids that copy. A returned long long is also a small scalar result.

Q2 – Return by Reference and Its Dangers (4 Marks)

```
int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
// INVALID: returning a reference to a local variable
int& unsafeMark() {
    int local = 90;
    return local; // dangling reference; undefined behaviour if used
}
```

A reference returned by highestMark refers to an existing array element. The caller can modify it directly, including on the left side of assignment.

```
int marks[] = {72, 91, 84, 88};
highestMark(marks, 4) += 2;
std::cout << highestMark(marks, 4) << '\\n'; // 93
```

Returning a reference to local is invalid because the local variable is destroyed when the function returns. The returned reference dangles; reading or writing through it has undefined behaviour. Compilers commonly issue a warning such as “reference to local variable returned”, but exact wording depends on compiler.

Reference, const reference and copy

Return form	Caller can modify?	Use
int&	Yes	Expose an existing element for controlled modification.
const int&	No through this reference	Avoid copying while preventing modification.
int	Modifies only the returned copy	Safest when a small value is sufficient.

Changed condition: vector growth

A std::vector may reallocate its storage when it grows beyond capacity. Reallocation moves elements to a new memory block and invalidates references, pointers and iterators to its old elements. A reference obtained before a growth operation must not be used unless it is known that no reallocation occurred. Reserve capacity when appropriate, or retain an index and access the element again after growth.

Alternative index or pointer

```
int highestIndex(const int a[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (a[i] > a[best]) best = i;
    return best;
}
// index usage:
int i = highestIndex(marks, 4);
marks[i] += 2;
// pointer alternative:
```

```
int* highestPointer(int a[], int n) {
    return &a[highestIndex(a, n)];
}
```

Finding the maximum takes O(n) time and O(1) extra space for reference, index and pointer approaches. An index is usually easiest to preserve across vector reallocation because it can be used to access the element again, provided the logical element ordering has not changed. A pointer or reference into vector storage can become invalid after reallocation.

Q3 – Default Arguments and Function Overloading (6 Marks)

Why the original design is ambiguous

The original declarations overlap: charge(double), charge(double,bool) and charge(double,bool=false,bool=false). A call such as charge(2.0) can match more than one declaration because the third overload can fill omitted arguments using defaults, while the first overload is also an exact match. The compiler cannot choose a unique best candidate, so the call is ambiguous.

Redesigned overload set

```
#include <iostream>
struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};
double charge(int grams) {
    std::cout << "grams overload\\n";
    return grams * 0.02;
}
double charge(double kg, bool express = false,
              bool insured = false) {
    std::cout << "double kg overload\\n";
    double amount = kg * 50.0;
    if (express) amount += 30.0;
    if (insured) amount += 20.0;
    return amount;
}
double charge(float kg) {
    std::cout << "float kg overload\\n";
    return charge(static_cast<double>(kg));
}
double charge(const Parcel& p) {
    std::cout << "Parcel overload\\n";
    return charge(p.weightKg, p.express, p.insured);
}
int main() {
    Parcel p{2.5, true, false};
    charge(2);          // int exact match
    charge(2.0f);       // float exact match
    charge(2.0);        // double exact match
    charge(2.5, true);  // double overload; insured defaults false
    charge(2.5, false, true); // all arguments supplied
    charge(p);          // Parcel exact match
}
```

Six call predictions

Call	Selected function	Reason
charge(2)	charge(int)	Exact match for int.
charge(2.0f)	charge(float)	Exact match for float; avoids float-to-double conversion competing with other overloads.
charge(2.0)	charge(double,bool,bool)	Exact first argument; bool arguments supplied by defaults.

charge(2.5,true)	charge(double,bool,bool)	Exact matches for double and bool; insured defaults false.
charge(2.5,false,true)	charge(double,bool,bool)	All parameter types match.
charge(p)	charge(const Parcel&)	Exact match for Parcel reference.

Overload resolution generally prefers an exact match over a promotion, and a promotion over a standard conversion. If two candidates are equally good, the call is ambiguous. Default arguments are supplied from the right because a trailing parameter may be omitted only when it has a default; a required parameter cannot follow an omitted one.

Alternative: Parcel structure or template

```
struct ParcelOptions {
    double weightKg{};
    bool express{false};
    bool insured{false};
};
double charge(const ParcelOptions& p) {
    double total = p.weightKg * 50.0;
    if (p.express) total += 30.0;
    if (p.insured) total += 20.0;
    return total;
}
```

Using one ParcelOptions object avoids a long overload list and makes adding another option (for example fragile handling) straightforward. A function template could support multiple numeric weight types, but may require constraints and careful conversion rules. Overload resolution occurs at compile time and has no direct runtime cost. Passing a Parcel by const reference avoids copying its fields; passing by value copies the structure, typically O(1) for a small fixed structure. Adding a new option to a structure adds a field and updates the calculation; adding an overload may increase ambiguity risk.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

SavingsAccount class with fixed circular transaction history

```
#include <iostream>
#include <string>
class SavingsAccount {
private:
    long long accountNumber;
    long long balancePaise;
    long long transactions[10];
    int count;
    int next; // next slot to write; circular index

    void record(long long amount) { // defined inside class
        transactions[next] = amount;
        next = (next + 1) % 10;
        if (count < 10) ++count;
    }
public:
    SavingsAccount(long long number, long long opening)
        : accountNumber(number), balancePaise(opening),
          transactions{}, count(0), next(0) {}

    bool deposit(long long amount);   // defined outside
    bool withdraw(long long amount);  // defined outside
    void miniStatement() const;       // defined outside
};

bool SavingsAccount::deposit(long long amount) {
    if (amount <= 0) return false;
    balancePaise += amount;
    record(amount);
    return true;
}
bool SavingsAccount::withdraw(long long amount) {
    if (amount <= 0 || amount > balancePaise) return false;
    balancePaise -= amount;
    record(-amount);
    return true;
}
void SavingsAccount::miniStatement() const {
    std::cout << "Account: " << accountNumber
              << " Balance (paise): " << balancePaise << '\\n';
    std::cout << "Recent transactions (oldest to newest):\\n";
    int first = (next - count + 10) % 10;
    for (int i = 0; i < count; ++i)
        std::cout << transactions[(first + i) % 10] << '\\n';
}
int main() {
    SavingsAccount account(10001, 50000);
    account.deposit(10000);
    account.withdraw(5000);
    account.deposit(2500);
    account.miniStatement();
}
```

The array and count are private so external code cannot corrupt the transaction history or count. Public member functions provide controlled operations. Some member functions (constructor and record) are defined inside the class; deposit(), withdraw() and miniStatement() are defined outside using SavingsAccount:: scope resolution.

What happens on the 11th transaction?

Method	Insertion cost	Behaviour
Shift array	O(k), here O(10)	Move existing entries left and write newest at the end; older entry is discarded.
Circular index	O(1)	Overwrite the oldest slot and advance next modulo capacity.

For the requirement of retaining the last 10 transactions, the circular index is preferable because it avoids shifting. If the requirement changes to 50, change the capacity and modulo logic; a named constexpr capacity or std::array makes the change safer.

Alternative deque and circular buffer complexity

A std::deque can use push_back() and pop_front() to maintain a fixed-size history, with O(1) operations at the ends in the standard complexity model. A fixed circular buffer also inserts in O(1). Shifting k elements costs O(k) per new transaction. Each design stores O(k) transaction values; a fixed array/circular buffer has predictable contiguous storage, while deque has implementation overhead and segmented storage.

Q5 – Static Members and Arrays of Objects (7 Marks)

Static member design

The issued counter and capacity belong to the BusTicket class, not to an individual ticket, so they should be static. Each ticket has its own seat and passenger name, so these remain non-static. A static data member is declared in the class and, in the traditional out-of-class form used in this assignment, defined once outside the class.

A static member function has no this pointer because it can be called without an object. It can directly access static members but cannot directly access a non-static member because no particular object is available.

Prediction for Token program

```
101 103 104 4
```

Token::next begins at 100. Constructing counter[3] invokes the constructor three times, assigning IDs 101, 102 and 103. Constructing extra invokes it once more and assigns 104. issued() returns next - 100 = 4.

Completed BusTicket TODOs

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count() { return issued; }
    void print() const {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting")) << '\\n';
    }
};
int BusTicket::issued = 0;
int main() {
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected output:

```
Asha: Seat 1\nBala: Seat 2\nChitra: Seat 3\nDev: Waiting\nIssued: 3
```

Arrays of objects and repeated function calls

BusTicket trip[4] creates four objects and calls the constructor for each. The shared static issued count increments only for the first three because capacity is three. If tickets are created inside a function called twice, the static count persists between calls; it is not reset when the function returns. Therefore, the second call continues numbering from the count left by the first call. A new process starts the static data member at its initial value again.

Alternative global or TicketOffice counter

A global counter also provides one shared count and O(1) operations, but it is less encapsulated and can be changed by unrelated code. A TicketOffice object passed to each BusTicket can own the counter and is more flexible: two independent TicketOffice objects can manage two buses with separate capacities and counts. All approaches use O(1) time per ticket and one counter per shared office/class; a static class member naturally represents one bus-wide count.

Q6 – Friend Functions and Inline Functions (8 Marks)

Completed friend functions

```
#include <iostream>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return first > second ? first - second : second - first;
}
bool earlier(const Time& a, const Time& b) {
    if (a.h != b.h) return a.h < b.h;
    return a.m < b.m;
}
int main() {
    Time departure(9,45), arrival(13,10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha
              << earlier(arrival, departure) << '\\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight. Their absolute difference is 205 minutes. Arrival is not earlier than departure, so the Boolean result is false.

Why friend?

A friend function is not a member function, but the class grants it access to private and protected members. It is called like an ordinary function: minutesBetween(a,b), not a.minutesBetween(b). It is suitable when an operation treats two objects symmetrically, such as comparing two times. A member function has an implicit this object and is called through one object.

Making h and m public would remove encapsulation: any caller could assign invalid values such as hour 99 or minute 75. Private data with controlled construction and operations protects the class invariant.

Correctness tests

Test	Expected result	Reason
09:45 and 13:10	205; earlier(13:10,09:45) = false	205-minute difference.
12:05 and 11:55	10; earlier(11:55,12:05) = true	Crosses noon; compare hour then minute.
08:30 and 08:30	0; earlier = false	Same time is not earlier.

Inline meaning and timing experiment

The inline keyword permits a function definition in a header and expresses an inline substitution preference to the compiler; it does not force the compiler to replace every call with the function body. Modern optimising compilers decide whether to inline based on optimisation and code size. Small functions such as a simple comparison may be good candidates. Large functions may increase code size if expanded at many call sites.

```
#include <chrono>
#include <iostream>
inline bool lessInline(int a, int b) { return a < b; }
bool lessNormal(int a, int b) { return a < b; }

int main() {
    volatile int sink = 0;
    const int N = 10000000;
    auto start1 = std::chrono::steady_clock::now();
    for (int i=0; i<N; ++i) sink += lessInline(i, i+1);
    auto end1 = std::chrono::steady_clock::now();

    auto start2 = std::chrono::steady_clock::now();
    for (int i=0; i<N; ++i) sink += lessNormal(i, i+1);
    auto end2 = std::chrono::steady_clock::now();

    std::cout << "sink=" << sink << '\\n';
    std::cout << "inline ns="
      << std::chrono::duration_cast<std::chrono::nanoseconds>(end1-start1).count()
      << '\\n';
    std::cout << "normal ns="
      << std::chrono::duration_cast<std::chrono::nanoseconds>(end2-start2).count()
      << '\\n';
}
```

Compile and run the benchmark twice: g++ -std=c++17 -O0 timing.cpp -o timing_O0 and g++ -std=c++17 -O2 timing.cpp -o timing_O2. Record actual elapsed values from your machine. At -O0 the call overhead may be visible; at -O2 the compiler may inline or otherwise optimise both functions, so measured differences may be small or absent. The volatile sink discourages removal of the loop, but this is still a microbenchmark and should be repeated several times.

Deployment decision and alternatives

Friend functions are justified if direct symmetric access to private state is needed and the interface is small. An alternative is member functions (a.minutesTo(b), a.isEarlier(b)) or public getters. Getters preserve private data but expose read access; member functions keep the operation associated with Time. All approaches take O(1) time per call and O(1) space. After inlining, their asymptotic cost is the same. Prefer member functions for behaviour naturally owned by one Time object; retain friend functions for symmetric operations that genuinely need private access.

Q7 – Diagnose, Redesign and Validate – Day-4 Design Challenge (13 Marks)

Selected limitation: shifting transaction history

The fixed-array SavingsAccount design in Q4 can maintain the last 10 transactions by shifting the remaining entries each time a new transaction arrives. The insertion work grows with the number of retained entries. When the history grows from 10 to 50, each new transaction may require moving up to 49 old entries. This is a design limitation caused by maintaining chronological order through repeated array shifting.

A. Diagnosis supported by complexity

For capacity k, shifting requires O(k) assignments per insertion. At k=10, at most 9 old entries are moved; at k=50, at most 49 are moved. A circular buffer writes the new transaction into one slot and advances an index, requiring O(1) work independent of k. The exact time in microseconds must be measured on the student's machine; the operation counts above are analytically derived.

B. Measurable objective

Redesign transaction insertion so that each deposit or withdrawal performs a constant number of transaction-buffer writes regardless of whether the retained history capacity is 10 or 50. Preserve the requirement that only the newest k transactions are available in chronological order.

C. Revised design and class sketch

```
SavingsAccount\n  - balancePaise\n  - transactions[k]\n  - count\n  - nextWriteIndex\n  +
deposit(amount)\n  + withdraw(amount)\n  + miniStatement()\n  + record(amount): write at
nextWriteIndex; advance modulo k
```

The revised design uses a circular array. It needs no extra dynamic allocation; it stores k transaction values, one count and one index. The trade-off is that miniStatement() must calculate the oldest element and wrap around the array when printing. Insertion becomes O(1), while printing the retained history remains O(k).

D. Prediction before implementation

Prediction: for each transaction, the circular implementation will perform one buffer assignment and one modulo/index update, while the shifting implementation performs up to k−1 assignments. At larger k, the circular implementation should show lower insertion work. Both use O(k) storage.

E. Implement and validate

```
#include <iostream>
#include <vector>
#include <chrono>


class CircularHistory {
    std::vector<long long> data;
    std::size_t next = 0, count = 0;
public:
    explicit CircularHistory(std::size_t capacity) : data(capacity) {}
    void add(long long value) {
        data[next] = value;
        next = (next + 1) % data.size();
```

```
        if (count < data.size()) ++count;
    }
    void print() const {
        std::size_t first = (next + data.size() - count) % data.size();
        for (std::size_t i=0; i<count; ++i)
            std::cout << data[(first+i)%data.size()] << ' ';
        std::cout << '\\n';
    }
};
class ShiftHistory {
    std::vector<long long> data;
    std::size_t count = 0;
public:
    explicit ShiftHistory(std::size_t capacity) : data(capacity) {}
    void add(long long value) {
        if (count < data.size()) data[count++] = value;
        else {
            for (std::size_t i=1; i<data.size(); ++i)
                data[i-1] = data[i];
            data.back() = value;
        }
    }
};
int main() {
    CircularHistory circular(5);
    for (int x=1; x<=7; ++x) circular.add(x);
    std::cout << "Circular last five: ";
    circular.print();

    const int N = 1000000;
    ShiftHistory shifting(50);
    CircularHistory ring(50);
    auto a = std::chrono::steady_clock::now();
    for (int i=0; i<N; ++i) shifting.add(i);
    auto b = std::chrono::steady_clock::now();
    for (int i=0; i<N; ++i) ring.add(i);
    auto c = std::chrono::steady_clock::now();
    std::cout << "Shift ns: "
      << std::chrono::duration_cast<std::chrono::nanoseconds>(b-a).count()
      << '\\n';
    std::cout << "Circular ns: "
      << std::chrono::duration_cast<std::chrono::nanoseconds>(c-b).count()
      << '\\n';
}
```

Functional expected output for the correctness check: adding 1 through 7 to capacity 5 retains 3, 4, 5, 6, 7 in chronological order. Timing numbers are machine-dependent and must be taken from actual execution; do not copy invented timing values.

```
Circular last five: 3 4 5 6 7
```

F. Defence and trade-offs

Criterion	Shifting array	Circular buffer
Insertion	O(k) when full	O(1)
History storage	O(k)	O(k)
Chronological display	Direct after shift	Requires wrap-around index

		calculation
Implementation	Simple indexing	Slightly more index logic
Best use	Very small history or infrequent updates	Frequent updates and larger fixed history

The objective is achieved at the algorithmic level because insertion no longer moves existing transactions. Actual timing should be compared using the same compiler, optimisation flags, capacity and number of insertions, with repeated runs. The original shifting design may remain preferable for a tiny history when simplicity is more important than insertion cost. The circular buffer is preferable when updates are frequent or capacity increases to 50.